

Day 1 Foundations & First Deployment

Participant Manual

Platform	AxisPay (fictional)
Kubernetes	v1.36
Environment	Ubuntu 26.04 LTS • Minikube
Built	11 August 2026

Every merchant, customer, card token, acquirer and transaction in this manual is fictional. No real institution is represented and no card number exists anywhere in this platform — only tokens. The platform is **not** PCI-DSS compliant and is not presented as such; it uses PCI-shaped constraints to make security controls consequential in a training context.

Day 4 — Networking, Exposure and Placement

AxisPay · Kubernetes Comprehensive · Participant Manual, Chapter 4

What changed today

Yesterday	Today
Reachable only via <code>port - forward</code>	Two hostnames over TLS
Flat pod network — the DMZ could read the database	22 NetworkPolicies, zero trust
Replicas placed by luck	One per node, by instruction
A drain evicted everything at once	Six PodDisruptionBudgets
DNS a black box	<code>ndots</code> , search domains and CoreDNS opened up

4.1 The cluster network model

1. What it is

A set of rules Kubernetes requires any network plugin to satisfy, plus the plugin (Calico, Cilium, Flannel) that actually satisfies them.

2. Why it exists

Kubernetes deliberately does not implement networking. It defines a contract — the **CNI** — so the same manifests run on a laptop, a data centre and three different clouds.

3. The business problem

AxisPay's services must reach each other predictably across nodes, and merchants must reach the platform from outside. Neither is possible without knowing what the network guarantees.

4. How it works

The four rules:

1. Every pod gets its own IP address — not a port on a shared host IP.
2. Pods reach all pods **without NAT**, across nodes.
3. Nodes reach all pods without NAT.
4. The IP a pod sees itself as is the IP others see it as.

Rule 4 sounds obvious and is not true of Docker's default bridge, where a container sees `172.17.0.x` while the world reaches it on a mapped host port. A container there cannot truthfully tell anyone its own address. Kubernetes forbids that, which is why service discovery is straightforward here.

5. Internal architecture

```
pod A --> node kernel --> CNI (Calico) --> node kernel --> pod B
                                routes/overlay
```

Calico can operate in **BGP mode** (real routes, no encapsulation, fastest) or **VXLAN/IPIP mode** (encapsulation, works anywhere). Minikube uses the latter.

Crucially, connectivity and policy enforcement are separate features. A plugin can satisfy all four rules and implement no NetworkPolicy at all — in which case every policy you write applies cleanly and enforces nothing.

6. Component interactions

```
kubelet -> CNI ADD -> plugin assigns an IP, wires the veth pair
kube-proxy -> watches EndpointSlices -> programs iptables/IPVS
Calico Felix -> watches NetworkPolicy -> programs additional rules
```

7. Enterprise example

A bank runs Calico in BGP mode peering with the physical network, so pod IPs are routable from outside the cluster. That makes firewall rules and traffic capture behave normally — worth a great deal to a security team, and impossible with an overlay.

8. Real-world analogy

A postal system. Every address is unique and reachable directly, and the letter arrives with the real sender's address on it. Contrast with a company switchboard that rewrites every outbound number — the recipient cannot call you back.

9. Best practices

Practice	Reason
Choose a CNI that enforces NetworkPolicy	Otherwise every policy you write is decoration
Choose it at cluster creation	CNI cannot be changed on a running cluster
Understand overlay vs routed mode	Overlay costs MTU and a little CPU; routed needs network cooperation
Watch MTU	An overlay reduces the usable MTU; mismatches cause slow, intermittent failures

10. Common mistakes

Mistake	Symptom
Assuming the default CNI enforces policy	Policies apply, nothing is enforced, no error
Ignoring MTU with an overlay	Large payloads hang; small ones work
Expecting pod IPs to be stable	They are not — that is what Services are for
Assuming pod IPs are routable outside	Only in routed mode

11. Security considerations

Threat	Control	Residual risk
Lateral movement across a flat network	NetworkPolicy + a policy-capable CNI	Requires the right plugin
Traffic sniffing between pods	Encryption (WireGuard in Calico, mTLS in a mesh)	Adds operational cost
Pod IP spoofing	Most CNIs enforce source addresses	Depends on the plugin

12. Performance considerations

- **Overlay encapsulation** costs a few percent CPU and reduces MTU (typically 1450 vs 1500).
- **iptables mode** in kube-proxy is $O(n)$ in rule count and degrades past a few thousand Services; **IPVS** uses hash tables.
- **DNS is frequently the real latency cost** — see §4.2.

13. High availability

The network is per-node and inherently distributed. Failure modes: a CNI DaemonSet pod unhealthy on one node (pods there cannot get IPs — the node goes `NotReady`), or a control-plane outage stopping *changes* while existing traffic continues.

14. Disaster recovery

CNI configuration lives on the node and in the plugin's own CRDs. Losing it means pods cannot get addresses. Back up Calico's IP pool configuration alongside etcd.

15. Monitoring

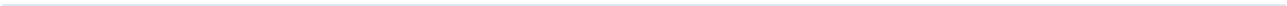
Metric	Why
<code>kube_node_status_condition{condition="NetworkUnavailable"}</code>	CNI failure on a node
CNI DaemonSet ready count	Should equal the node count
<code>kubeproxy_sync_proxy_rules_duration_seconds</code>	Rising = too many Services
Pod IP allocation failures	IP pool exhaustion

16. Troubleshooting

Symptom	Cause	Command	Fix
Node <code>NotReady</code> , <code>NetworkUnavailable</code>	CNI pod unhealthy	<code>kubectl get pods -n kube-system -l k8s-app=calico-node -o wide</code>	Restart the CNI pod
Pod stuck <code>ContainerCreating</code>	CNI cannot assign an IP	<code>kubectl describe pod → Events</code>	Check the IP pool
Cross-node traffic fails, same-node works	Overlay/routing broken	Test with pod IPs directly	Check the CNI and MTU
Large requests hang, small ones work	MTU mismatch	<code>ping -M do -s 1400</code> between pods	Align MTU
NetworkPolicies do nothing	CNI does not enforce them	<code>kubectl get ds -n kube-system</code>	Rebuild with a policy-capable CNI

Interview questions

1. **What are the four networking rules Kubernetes requires?** *Every pod has its own IP; pods reach all pods without NAT; nodes reach all pods without NAT; the IP a pod sees itself as is the one others see.*
2. **Does Kubernetes implement networking?** *No. It defines the CNI contract and a plugin implements it. That is why the plugin choice matters and why it cannot be changed on a running cluster.*
3. **Your NetworkPolicies apply cleanly and traffic still flows. Why?** *(senior) The CNI does not implement policy enforcement. Connectivity and policy are separate features; Kubernetes stores the object regardless. Verify with a policy-capable CNI such as Calico or Cilium.*



4.2 DNS and service discovery

1. What it is

CoreDNS: a cluster DNS server that answers queries for Services and pods.

2. Why it exists

Pod IPs change constantly. Services provide a stable IP; DNS provides a stable *name* so configuration does not carry addresses.

3. The business problem

Every AxisPay manifest configures downstream services by name. Those names must resolve in every namespace, and the resolution must be fast enough to sit inside a 300 ms budget.

4. How it works

Every pod gets `/etc/resolv.conf`:

```
nameserver 10.96.0.10
search axispay-core.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

Form	Resolves from
<code>payment-service</code>	Same namespace only
<code>payment-service.axispay-core</code>	Anywhere
<code>payment-service.axispay-core.svc.cluster.local</code>	Anywhere — fully qualified
<code>postgres-0.postgres.axispay-data.svc.cluster.local</code>	A specific pod of a StatefulSet

ndots:5 is the performance trap. A name with fewer than five dots gets every search suffix appended *before* being tried as an absolute name. `payment-service.axispay-core.svc` has three dots, so the resolver tries up to four wrong names first. A **trailing dot** makes it absolute and skips the search list entirely.

5. Internal architecture

CoreDNS is a Deployment (usually 2 replicas) behind a ClusterIP Service. It watches the API server for Services and EndpointSlices and answers from memory. Its behaviour is configured by the **Corefile** in a ConfigMap — and a typo there crash-loops both replicas, which is INC-4b.

6. Component interactions

```
pod -> resolv.conf -> kube-dns ClusterIP -> (kube-proxy rules) -> CoreDNS pod
CoreDNS -> watches API server -> answers from cache
```

Note the recursion: reaching CoreDNS depends on kube-proxy having programmed the rules for the `kube-dns` Service.

7. Enterprise example

A high-throughput platform deploys **NodeLocal DNSCache** — a DaemonSet caching DNS on every node — cutting p99 lookup latency and removing CoreDNS as a hot path. They also set `ndots: 2` cluster-wide and use FQDNs everywhere.

8. Real-world analogy

A phone book plus a habit of trying extensions before full numbers. `ndots: 5` is that habit: for short names it tries every local prefix before dialling the number as given.

9. Best practices

Practice	Reason
Use FQDNs in configuration	Removes cross-namespace ambiguity and avoids search-list overhead
Allow both UDP and TCP on 53 in policies	TCP is the fallback for large responses; UDP-only fails intermittently
Run at least 2 CoreDNS replicas	It is on every request path
Consider NodeLocal DNSCache at scale	Removes a hot dependency
Watch CoreDNS resource usage	It is CPU-bound under heavy query load

10. Common mistakes

Mistake	Symptom
Short names across namespaces	Resolution failure
Allowing only UDP/53 in a policy	Intermittent failures — much harder to diagnose
Forgetting DNS egress after default-deny	Everything fails with a resolution error
Ignoring <code>ndots</code>	Three wasted round-trips per call at scale
Single CoreDNS replica	One restart takes out cluster-wide resolution

11. Security considerations

Threat	Control	Residual risk
DNS enumeration reveals architecture	NetworkPolicy; DNS is broadly readable in-cluster	Hard to prevent entirely
DNS spoofing	Cluster DNS is internal; use mTLS for real authentication	—
DNS as an exfiltration channel	Egress policy restricting external DNS	Internal DNS must stay open

12. Performance considerations

- `ndots:5` costs up to **four extra lookups** per short name.
- CoreDNS caches; a cold cache after a restart shows as a latency spike.
- DNS failures often appear *gradually* because cached entries expire at different times — which is exactly why INC-4b presents as "intermittent".

13. High availability

Two replicas, anti-affinity across nodes, and a PDB. NodeLocal DNSCache adds a per-node layer so a CoreDNS outage degrades rather than breaks.

14. Disaster recovery

CoreDNS is stateless; recovery is redeploying it. The Corefile is the thing to back up — and to review carefully, since a typo takes out cluster-wide resolution.

15. Monitoring

Metric	Threshold
<code>coredns_dns_responses_total{rcode="SERVFAIL"}</code>	Any sustained rate
<code>coredns_dns_request_duration_seconds p99</code>	> 50 ms
CoreDNS ready replicas	< 2
<code>coredns_cache_hits_total</code> / misses	Falling hit rate = cache pressure

16. Troubleshooting

Symptom	Cause	Command	Fix
Nothing resolves	CoreDNS down	<code>kubectl get pods -n kube-system -l k8s-app=kube-dns</code>	Restart; check the Corefile
Short name fails cross-namespace	Expected	—	Use the FQDN
Intermittent failures	Only UDP/53 allowed, or CoreDNS flapping	<code>kubectl get netpol -o yaml ;</code> CoreDNS logs	Allow TCP/53
Slow DNS under load	<code>ndots:5</code> with short names	<code>time nslookup</code>	FQDNs, trailing dots, or NodeLocal cache
Resolution fails only after a policy change	DNS egress blocked	<code>kubectl get netpol</code>	Add <code>allow-dns-egress</code>

Interview questions

1. **What does `ndots:5` do?** Any name with fewer than five dots has the search suffixes appended before being tried as absolute — up to four wasted lookups. A trailing dot skips the search list.
2. **Why allow both UDP and TCP on port 53?** DNS uses UDP normally and falls back to TCP for responses over 512 bytes. Allowing only UDP produces intermittent failures that are far harder to diagnose than a total outage.
3. **You apply a default-deny NetworkPolicy and everything breaks. Why?** (senior) Every service call begins with a DNS lookup to CoreDNS in `kube-system`, and that lookup is egress traffic. The symptom is name resolution failure, not connection refusal, so it looks like a DNS outage rather than a policy problem.

4.3 Ingress

1. What it is

Two things: an **Ingress resource** describing HTTP routing rules, and an **Ingress controller** that reads those rules and proxies traffic.

2. Why it exists

A LoadBalancer Service per application is expensive and gives you no HTTP-level routing. Ingress puts one entry point in front of many services, with host and path routing plus TLS.

3. The business problem

Kalahari Coffee Roasters go live on Monday and will call `https://api.axispay.local/api/v1/charges` from their own servers. `kubect1 port-forward` is not a product, and card data cannot travel over plaintext HTTP.

4. How it works

```
merchant --TLS:443--> Ingress controller --http:8080--> Service --> pods
                        ^
                    reads Ingress resources
```

An Ingress with no controller is a document nobody reads. It appears in `kubect1 get ingress` with an **empty ADDRESS**, and nothing works. That empty column is the diagnostic.

`pathType` decides matching:

Type	<code>/api/v1</code> matches
Prefix	<code>/api/v1</code> , <code>/api/v1/charges</code> , everything below
Exact	only the literal <code>/api/v1</code>
ImplementationSpecific	controller-dependent — avoid

5. Internal architecture

`ingress-nginx` runs as a Deployment, watches Ingress resources, generates an nginx configuration and reloads.

`ingressClassName` decides which controller claims a resource — essential on a cluster with more than one.

Controller-specific behaviour arrives via **annotations** (`limit-rps`, `proxy-read-timeout`, `ssl-redirect`), which is the main criticism of Ingress: the portable part is small and the useful part is vendor-specific. **Gateway API** is the successor and addresses exactly this.

6. Component interactions

```
you      -> create Ingress
controller -> watch -> regenerate nginx.conf -> reload
merchant -> TLS to controller -> terminate -> proxy to Service ClusterIP
Service  -> EndpointSlice -> pod
```

7. Enterprise example

A payments platform runs two ingress controllers: an internet-facing one for merchant APIs with strict rate limits and WAF integration, and an internal one for staff tools. `ingressClassName` separates them, and certificates are issued by cert-manager against an ACME issuer with automatic rotation.

8. Real-world analogy

A building receptionist. One front door, one set of rules about who goes where, and one place where identity is checked. Without a receptionist the door still exists — nobody is directing anyone.

9. Best practices

Practice	Reason
Always set <code>ingressClassName</code>	Otherwise nobody claims it, or two controllers both do
Use <code>pathType: Prefix</code> unless you mean exact	<code>Exact</code> matches one literal path
Terminate TLS at the Ingress	One place for certificates
Use cert-manager in production	Manual certificates expire — and INC-7 is exactly that
Rate limit at the edge	Cheaper than at the application
Never expose <code>/healthz</code> externally	Probe endpoints are internal
Read the controller's logs when debugging	They record the routing decision

10. Common mistakes

Mistake	Symptom
No controller installed	Empty ADDRESS, nothing works, no error
<code>pathType: Exact</code> by accident	404 on every endpoint but one — INC-4a
Wrong backend port name	502
Backend Service with no endpoints	503
Forgetting the NetworkPolicy for the controller	503 after applying default-deny
Certificate expiring unnoticed	Total outage — INC-7

11. Security considerations

Threat	Control	Residual risk
Plaintext card data	TLS termination + <code>ssl-redirect</code>	TLS ends at the Ingress; use mTLS inside for defence in depth
Volumetric abuse	Rate limiting annotations	A distributed flood needs upstream help
Certificate expiry	cert-manager with automatic renewal	Monitoring is still required
Exposing internal endpoints	Explicit path rules; never wildcard	Review changes carefully

12. Performance considerations

- TLS termination is CPU-bound; scale the controller with traffic.
- **nginx reloads on every Ingress change** — thousands of Ingresses make reloads slow.
- `proxy-read-timeout` must exceed your slowest legitimate request; the reporting Ingress uses 120 s against the API's 30 s.

13. High availability

Run several controller replicas with anti-affinity and a PDB. In cloud environments they sit behind a LoadBalancer Service. The controller is a single point of failure for **all** external traffic — treat it accordingly.

14. Disaster recovery

Ingress resources are configuration and recover from Git. **Certificates are the hard part**: manual ones must be backed up and restored, which is precisely why cert-manager is the production answer. Also note that recreating a cloud LoadBalancer usually allocates a **new IP**, breaking DNS and any merchant allow-lists.

15. Monitoring

Metric	Threshold
<code>nginx_ingress_controller_requests{status=~"5.."} rate</code>	Any increase
<code>nginx_ingress_controller_ssl_expire_time_seconds</code>	< 30 days — page
Controller ready replicas	< 2
<code>nginx_ingress_controller_config_last_reload_successful</code>	0 = bad configuration

16. Troubleshooting

Symptom	Cause	Command	Fix
ADDRESS empty	No controller, or wrong class	<code>kubectl get ingressclass</code>	Install; set <code>ingressClassName</code>
404	Path or <code>pathType</code> wrong	Controller logs	Use <code>Prefix</code>
502	Wrong backend port	<code>kubectl describe ingress</code>	Match the Service port name
503	Backend has no endpoints	<code>kubectl get endpointslice</code>	Fix the Service or readiness
503 only after policy changes	Controller blocked by NetworkPolicy	<code>kubectl get netpol -n <ns></code>	Allow the <code>ingress-nginx</code> namespace
TLS warning	Self-signed	<code>openssl s_client</code>	cert-manager

404, 502 and 503 point at three different layers. Knowing which saves ten minutes every time.

Interview questions

1. **What is the difference between an Ingress resource and an Ingress controller?** *The resource is a document describing rules; the controller is a program that reads it and proxies. Creating a resource with no controller changes nothing — `kubectl get ingress` shows an empty ADDRESS.*
2. **Prefix versus Exact ?** *Prefix matches the path and everything below it. Exact matches only the literal path. Changing one word turns a working API into a 404 on every endpoint but one.*
3. **You get 404, then 502, then 503 from an Ingress. What does each tell you?** *(senior) 404: the routing rules did not match — an Ingress problem. 502: the controller reached a backend and got a bad response, usually the wrong port. 503: no ready endpoints behind the Service — a Service or readiness problem. Three codes, three layers.*
4. **What does Gateway API fix?** *(senior) It replaces controller-specific annotations with typed resources, separates infrastructure and application concerns into different objects with different RBAC, and supports protocols beyond HTTP. Ingress's portable surface is small and its useful surface is vendor-specific; Gateway API addresses both.*

4.4 NetworkPolicy

1. What it is

A namespaced object describing which traffic is permitted to and from a set of pods.

2. Why it exists

By default every pod can reach every other pod in the cluster. In a regulated environment that is a finding, not a feature.

3. The business problem

Yesterday you connected to PostgreSQL from `edge-gateway`. In a PCI assessment that puts the DMZ inside the **cardholder data environment** — so every CDE control applies to the gateway, to everyone who can deploy it, and to its logs. The audit gets larger, longer and more expensive.

4. How it works

Three properties decide everything:

1. **Default-allow until selected.** A pod with no policy selecting it is unrestricted. The moment *any* policy selects it, only what a policy explicitly permits is allowed.
2. **Additive — there is no deny rule.** Policies only add permission. You deny by selecting a pod and permitting nothing.
3. **Both directions, independently.** Egress at the source *and* ingress at the destination must both allow a flow — two objects, often in two namespaces.

Default-deny is therefore just:

```
spec:
  podSelector: {}           # every pod
  policyTypes: [Ingress, Egress] # both directions
  # ...and no rules at all
```

Enforcement is the CNI's job. Kubernetes stores the object whatever your plugin does.

5. Internal architecture

AxisPay's 22 policies, in the order they are applied:

#	Policy	Effect
1	<code>default-deny-all</code> × 4 namespaces	Everything stops
2	<code>allow-dns-egress</code> × 4	UDP and TCP on 53 to CoreDNS
3	<code>allow-edge-to-payment</code> / <code>-merchant</code>	The DMZ may enter the CDE, narrowly
4	<code>allow-payment-to-core-services</code>	Only <code>payment-service</code> may call fraud/routing/ledger/customer
5	<code>allow-core-to-data</code>	Core reaches PostgreSQL, Redis, RabbitMQ
6	<code>allow-core-and-async-to-data</code>	The vault accepts core and async — not edge
7	<code>allow-ingress-controller</code>	ingress-nginx may reach the gateway
8	<code>allow-prometheus-scrape</code> × 3	Observability may scrape; nothing may reach it

The most important policy is the one that does not exist. Nothing grants `axispay-edge` access to `axispay-data`. That omission is the control.

6. Component interactions

```
you    -> create NetworkPolicy
API    -> store
Calico Felix -> watch -> program iptables/eBPF on every node
packet -> evaluated at BOTH ends: egress at source, ingress at destination
```

7. Enterprise example

A bank runs default-deny in every namespace, generated automatically at namespace creation. Policy changes require a review from the security team, and the CI pipeline runs a policy simulator against a fixed list of must-allow and must-block flows before merge. That simulator is `scripts/validate/simulate-netpol.py` in this repository.

8. Real-world analogy

Building access control where **every door is locked by default** and each badge lists exactly which doors it opens. There is no "deny" badge — access is the absence of permission.

Where it breaks: a building has one access system. Kubernetes evaluates the door you leave *and* the door you enter, independently, and both must permit you.

9. Best practices

Practice	Reason
Default-deny first, then allow-list	Building the allow-list first looks strict and enforces nothing
Always allow DNS egress explicitly	Otherwise everything fails with a resolution error
Allow both UDP and TCP on 53	UDP-only gives intermittent failures
Write egress rules, not just ingress	Ingress stops attackers getting in; egress stops data getting out
Test enforcement empirically	Reading YAML proves nothing
Simulate in CI	A silently non-enforcing policy set is invisible
Label namespaces deliberately	<code>namespaceSelector</code> matches labels, not names

10. Common mistakes

Mistake	Symptom
Forgetting DNS egress	Everything fails; looks like a DNS outage
Only allowing UDP/53	Intermittent failures
Writing only ingress rules	No exfiltration protection
Assuming a policy denies	Policies only add; absence denies
Adding a narrow policy to an open pod	Everything else to that pod is now blocked — INC-4c
CNI without enforcement	Applies cleanly, protects nothing
Deleting a policy to fix an outage	Service restored, control removed, audit failed

11. Security considerations

Threat	Control	Residual risk
Lateral movement after a compromise	Default-deny + narrow allow-list	A compromised pod can still use its own permitted paths
Data exfiltration	Egress rules	DNS egress must stay open and is a known covert channel
DMZ reaching cardholder data	No policy grants it — the omission	Someone can add one
Policy silently not enforced	Verify the CNI; simulate in CI	Requires discipline

12. Performance considerations

Policies become iptables or eBPF rules. Very large policy sets increase rule-evaluation cost; Calico's eBPF mode scales better than iptables. Policy changes are pushed to every node, so churn costs CPU across the cluster.

13. High availability

Policies are enforced per node by the CNI agent. If that agent is unhealthy on a node, enforcement there may be stale — which is a **security** failure rather than an availability one, and far less visible. Monitor CNI agent health as a security control.

14. Disaster recovery

Policies are configuration and recover from Git. The operational risk is the opposite of loss: someone **deleting** a policy during an incident to restore service, and it never being restored. Any policy change made under pressure must be reviewed afterwards.

15. Monitoring

Signal	Why
Calico denied-packet metrics	Shows what policy is dropping
CNI agent ready count	Stale enforcement is silent
Policy count per namespace	A sudden drop means someone deleted one
Simulator result in CI	Catches regressions before merge

16. Troubleshooting

Symptom	Cause	Command	Fix
Everything fails after default-deny	DNS egress missing	<code>nslookup</code> from a pod	Add <code>allow-dns-egress</code>
Intermittent failures	UDP-only DNS rule	<code>kubectl get netpol -o yaml</code>	Allow TCP/53
Applied, nothing enforced	CNI lacks policy support	<code>kubectl get ds -n kube-system</code>	Rebuild with Calico
Traffic blocked, no error anywhere	Packets dropped, not refused	<code>nc -z -w3</code> from a pod	Read the policies — INC-4c
Policy looks right, still blocked	Only one direction allowed	Check both namespaces	Both must permit
<code>namespaceSelector</code> matches nothing	Namespace missing the label	<code>kubectl get ns --show-labels</code>	Label it

Interview questions

1. **Is NetworkPolicy default-allow or default-deny?** *Default-allow, until at least one policy selects the pod. From then on, only explicitly permitted traffic is allowed.*
2. **How do you write a deny rule?** *You cannot. There is no deny. You ensure some policy selects the pod and that none of them permit the traffic. Absence of permission is the denial.*

3. **You apply default-deny and everything breaks, including calls you allowed. Why?** *DNS. Every service call starts with a lookup to CoreDNS, which is egress traffic. The symptom is name resolution failure, so it reads as a DNS outage rather than a policy problem.*
 4. **A narrow policy is added to one service and a completely different service breaks. Explain.** *(senior) Policies are additive and default-allow-until-selected. Before the policy, the target pod accepted traffic from anywhere. Adding one narrow policy makes it accept traffic ONLY from what that policy permits — everything else is now denied, including callers never mentioned in it. That is INC-4c.*
 5. **A QSA asks you to demonstrate that the DMZ cannot reach cardholder data. What do you show?** *(senior) A live connection attempt from a DMZ pod to the database that times out, the policy set showing no rule granting that path, and a CI-run simulator asserting it. Evidence is a failed connection and a repeatable test — not a YAML file.*
-

4.5 Placement and disruption

1. What it is

Controls over **where** pods run (nodeSelector, affinity, taints) and **how many** may be voluntarily removed at once (PodDisruptionBudget).

2. Why it exists

Three replicas on one node is not redundancy — it looks like it and is not. And a node drain with no budget evicts everything on it simultaneously.

3. The business problem

Friday's capstone upgrades AxisPay under live merchant traffic, and part of that is draining a node. Without budgets that is a visible outage.

4. How it works

Three mechanisms, three directions:

Mechanism	Direction
<code>nodeSelector</code> / <code>nodeAffinity</code>	The pod is attracted to certain nodes
<code>podAffinity</code> / <code>podAntiAffinity</code>	The pod is attracted to / repelled by other pods
taints / tolerations	The node repels pods

A **toleration does not attract**. It only removes an objection. To repel everything else *and* attract a specific workload you need a taint on the node plus both a toleration and a `nodeSelector` on the pod.

required vs preferred :

	Behaviour	AxisPay uses it for
<code>requiredDuringScheduling...</code>	Hard filter. Unsatisfiable → Pending forever.	<code>payment-service</code> — one per node, absolutely
<code>preferredDuringScheduling...</code>	Scoring hint, best effort	<code>fraud-service</code> — its HPA scales past the node count

The trap: `required` anti-affinity on hostname means you can never have more replicas than nodes. Combined with an HPA it silently caps autoscaling — and the symptom (pods Pending during a spike) looks nothing like the cause.

PodDisruptionBudget gates **voluntary** disruption only:

Event	PDB applies?
<code>kubectl drain</code>	Yes
Cluster/node-pool upgrade	Yes
Node crash	No
OOM kill	No
Liveness restart	No
<code>kubectl delete pod</code>	No — a direct delete bypasses the eviction API

5. Internal architecture

The scheduler runs **filter** then **score**. Filter removes nodes that cannot run the pod (resources, taints, `required` affinity, node selectors). Score ranks the survivors (spread, `preferred` affinity, image locality). If filtering leaves nothing, the pod stays `Pending`.

`topologySpreadConstraints` are more expressive than anti-affinity: `maxSkew` allows "roughly even within a tolerance" rather than binary same-node-or-not, and `whenUnsatisfiable` chooses between refusing and degrading.

6. Component interactions

```

drain -> cordon node -> evict pods one at a time
      -> eviction API consults the PDB
      -> if the budget would be breached: WAIT and retry
      -> replacement schedules elsewhere (affinity applies again)

```

7. Enterprise example

A bank spreads every production workload across three availability zones with `topologySpreadConstraints` on `topology.kubernetes.io/zone`, and gives every workload a PDB of `maxUnavailable: 1`. Node-pool rotations run continuously and are a non-event. Nodes hosting cardholder workloads are tainted so nothing else can land on them.

8. Real-world analogy

Staff rostering. Anti-affinity is "not everyone on the same shift". A taint is "this site requires a security clearance". A toleration is "I have the clearance" — which does not mean you are assigned there. A PDB is "at least four people must be on the floor at all times", which constrains planned leave and does nothing about someone calling in sick.

9. Best practices

Practice	Reason
required anti-affinity for critical paths	Real redundancy, not apparent
preferred for autoscaled workloads	required silently caps the HPA
Prefer maxUnavailable over minAvailable	An absolute minAvailable can block all drains at minReplicas
Give every production workload a PDB	Otherwise a drain takes everything
Document the <i>absence</i> of a budget	A future operator needs to know it is deliberate
Spread on zone, not just hostname	A hostname spread does not survive a zone failure
Never maxUnavailable: 0	The node becomes undrainable, so it cannot be patched

10. Common mistakes

Mistake	Symptom
required anti-affinity plus an HPA	Autoscaling silently caps at the node count
minAvailable equal to current replicas	Drain hangs forever
maxUnavailable: 0	Node can never be maintained
Toleration without nodeSelector	Pod tolerates the taint but is not attracted there
No PDB	Drain evicts everything at once
Expecting a PDB to prevent a crash	It gates voluntary disruption only

11. Security considerations

Threat	Control	Residual risk
Cardholder workloads on shared nodes	Taints + nodeSelector for a PCI node pool	Kernel is still shared
An attacker forcing eviction	RBAC on pods/eviction	Cluster-admin can always drain
PDB used to block maintenance	Alert on disruptionsAllowed: 0	Requires monitoring

12. Performance considerations

- **required** affinity increases scheduling time — every candidate node is checked.
- **podAffinity** is expensive at scale: it is O(pods) per scheduling decision.
- **percentageOfNodesToScore** trades placement quality for speed on very large clusters.

13. High availability

Placement is the mechanism by which replica count becomes availability. Three replicas on one node survive nothing. The full recipe: replicas ≥ 3 · anti-affinity or topology spread · a PDB · readiness probes · storage that survives node loss.

14. Disaster recovery

Placement rules are configuration. The DR-relevant question is whether your constraints are **satisfiable in a degraded cluster**: `required` anti-affinity across three nodes means losing one node leaves a pod permanently Pending. That is correct behaviour, and it must be understood before an incident rather than discovered during one.

15. Monitoring

Metric	Threshold
<code>kube_poddisruptionbudget_status_pod_disruptions_allowed</code>	0 — maintenance is blocked
<code>kube_pod_status_unschedulable</code>	Any sustained
Pods per node for a critical Deployment	> 1 means spread failed
Node <code>unschedulable</code> (cordoned)	Longer than a maintenance window

16. Troubleshooting

Symptom	Cause	Command	Fix
Pod Pending, "didn't match pod anti-affinity"	More replicas than nodes with <code>required</code>	<code>kubectl describe pod</code>	Use <code>preferred</code> , or add nodes
HPA stops at the node count	<code>required</code> anti-affinity	<code>describe a Pending pod</code>	Switch to <code>preferred</code>
Drain hangs forever	PDB allows zero disruptions	<code>kubectl get pdb</code>	Scale up, or relax the budget
Drain evicts everything at once	No PDB	<code>kubectl get pdb -A</code>	Apply one
Toleration added, pod does not move	A toleration does not attract	<code>kubectl get pod -o wide</code>	Add a <code>nodeSelector</code>
Replicas still stacked	Rule applied but not rolled out	<code>kubectl rollout status</code>	Trigger a rollout

Interview questions

1. **What is the difference between a taint and anti-affinity?** *Direction.* A taint is the node repelling pods; anti-affinity is the pod repelling other pods. A toleration removes the node's objection but does not attract the pod — for that you also need a `nodeSelector`.

2. **When would you use `preferred` rather than `required` anti-affinity?** *When the replica count can exceed the node count — anything behind an HPA. `required` means you can never have more replicas than nodes, so it silently caps autoscaling during exactly the spike it was meant to absorb.*
 3. **Does a `PodDisruptionBudget` protect against a node crashing?** *No. It gates voluntary disruption only — drains and upgrades via the eviction API. A crash, an OOM kill, a liveness restart and even `kubectl delete pod` all bypass it.*
 4. **Your PDB is `minAvailable: 3` and the HPA has scaled to 3. You drain. What happens?** *(senior) Nothing, forever. Zero disruptions are allowed, so the drain hangs and the node can never be maintained. A node that cannot be drained cannot be patched — which is why `maxUnavailable` is safer with autoscaling.*
 5. **Design placement for a payment service that must survive an availability-zone failure.** *(senior) `topologySpreadConstraints` on `topology.kubernetes.io/zone` with `maxSkew: 1` and `DoNotSchedule`, at least one replica per zone, `minReplicas` high enough that losing a zone still leaves capacity, a PDB of `maxUnavailable: 1`, and storage that is either zone-redundant or replicated at the application level.*
-

Day 4 cheat sheet

Services and DNS

```
kubectl get svc,endpointslice -A
kubectl get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>

# the four forms
<svc>                                # same namespace only
<svc>.<ns>                            # anywhere
<svc>.<ns>.svc.cluster.local         # fully qualified – use this
<pod>-0.<svc>.<ns>.svc.cluster.local # a specific StatefulSet pod

kubectl exec <pod> -- cat /etc/resolv.conf    # ndots:5 lives here
```

Ingress

```
kubectl get ingress -A          # EMPTY ADDRESS = no controller claimed it
kubectl describe ingress <name> -n <ns>
kubectl logs -n ingress-nginx -l app.kubernetes.io/component=controller --tail=20
```

Code	Layer
404	Routing rules did not match — Ingress
502	Wrong backend port
503	No ready endpoints — Service or readiness

NetworkPolicy

```
kubectl get netpol -A
python3 scripts/validate/simulate-netpol.py    # 39 assertions

# test enforcement – reading YAML proves nothing
kubectl run probe -n <ns> --rm -i --restart=Never --image=busybox:1.37 -- \
  sh -c 'nc -z -w3 <host> <port> && echo ALLOWED || echo blocked'
```

Three properties: default-allow until selected · additive, no deny rule · both directions independently. **Always allow** DNS egress, UDP *and* TCP on 53.

Placement and disruption

```
kubectl get pods -o wide          # are replicas actually spread?
kubectl get pdb -A               # ALLOWED DISRUPTIONS must be > 0
kubectl describe node <n> | grep -A5 Taints
kubectl cordon <n>; kubectl drain <n> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <n>
```

	required	preferred
Unsatisfiable	Pending forever	Scheduled anyway
Use for	Critical paths	Anything autoscaled

Day 4 review questions

1. Name the four networking rules Kubernetes requires of a CNI.
2. Does Kubernetes implement networking? What implements NetworkPolicy?
3. What does `ndots:5` cost, and how do you avoid it?
4. Why must a NetworkPolicy allow both UDP and TCP on port 53?
5. What is the difference between an Ingress resource and an Ingress controller?
6. `pathType: Prefix` versus `Exact` — what does each match?
7. 404, 502 and 503 from an Ingress: what does each point at?
8. Is NetworkPolicy default-allow or default-deny?
9. How do you write a "deny" rule?
10. You apply default-deny and everything breaks. Why, and what is the symptom?
11. Adding one narrow policy broke a service not mentioned in it. Explain.
12. What is the difference between a taint and anti-affinity?
13. Does a toleration attract a pod to a node?
14. Why does `required` anti-affinity silently cap an HPA?
15. Does a PDB protect against a node crash? An OOM kill? `kubectl delete pod`?
16. Why prefer `maxUnavailable` over `minAvailable` with an HPA?
17. Your PDB allows zero disruptions. What is the operational consequence?
18. How would you prove to an auditor that the DMZ cannot reach cardholder data?

Answers: <documents/assessments/answer-keys/day4-answer-key.md>

Day 4 summary

You built: two TLS-terminated hostnames reachable from outside the cluster · **22 NetworkPolicies** enforcing zero trust across four namespaces · `payment-service` replicas one per node by instruction · six PodDisruptionBudgets · four async services deployed.

You proved: the four networking rules from your own cluster · that `ndots:5` costs real round-trips · that an Ingress with no controller does nothing and says nothing · that default-deny breaks DNS first · **that the DMZ can no longer reach the payments database** — the same command that opened the day, with the opposite result · that a node drains under 40 rps with zero failed payments.

What is still missing:

Gap	Consequence	Fixed
No RBAC	Every ServiceAccount can do anything	L5.2
Pod Security not enforced at namespace level	A workload that forgets <code>securityContext</code> is still admitted	L5.1
No packaging	Deploying is 40 <code>kubectl apply</code> commands	L5.3
No metrics or dashboards	You cannot operate what you cannot see	L5.5
No log aggregation	Tracing one payment means <code>kubectl logs</code> across 15 services	L5.6
No alerts	A merchant tells you before your monitoring does — as happened all week	L5.6

Tonight (optional, 10 minutes): run `python3 scripts/validate/simulate-netpol.py` and read the output. Then look at `MUST_BLOCK` and ask yourself which of those you would have thought to test.